Real-Time Object Recognition and Augmented Reality Overlay Using Deep Learning

Sangeeth Deleep Menon
Khoury College of Computer Sciences
Northeastern University
Boston, MA, USA
NUID: 002524579

Raj Gupta
Khoury College of Computer Sciences
Northeastern University
Boston, MA, USA
NUID: 002068701

***Abstract*—This paper presents a real-time markerless augmented reality system that uses deep learning to recognize everyday objects and overlay 3D graphics onto them through a live webcam feed. Three neural network architectures were implemented and compared: a three-layer convolutional neural network (CNN) trained from scratch, a minimal Vision Transformer (ViT) trained from scratch, and a MobileNetV2 backbone fine-tuned with transfer learning. A dataset of 1419 images across 10 object classes was assembled using a combination of webcam captures and web-downloaded images. The MobileNetV2 model achieved 88.8% test accuracy on the 10-class dataset and was selected as the deployed backbone. When the classifier detects an object with sufficient confidence, the system projects a 3D wireframe box onto the object using a fixed-depth pinhole camera model and renders a floating label tag showing the class name and confidence score. The complete pipeline runs at 20 to 30 frames per second on an Apple Silicon MacBook using MPS acceleration. The system requires no printed calibration target, making any recognized object serve as its own AR anchor.**

***Index Terms*—augmented reality, object recognition, convolutional neural network, vision transformer, transfer learning, MobileNetV2, real-time inference**

I. Introduction

Augmented reality (AR) systems traditionally rely on physical markers or calibration targets to estimate camera pose before rendering 3D graphics into a scene. While effective, this requirement limits where and how AR can be deployed in practice because the target must always be present and visible. An alternative approach is to use an object recognition system to detect and localize known objects directly, turning any recognized item into its own virtual anchor point.

This project explores exactly that idea. It builds a live pipeline that takes frames from a standard webcam, classifies the object in a center region of interest using a trained neural network, and draws a 3D wireframe box along with a floating label onto the detected object. The goal is a smooth, responsive demo that works on everyday items a person might have on their desk, such as a phone, keyboard, book, or cup, without any specialized hardware or printed calibration pattern.

The system brings together two main components. The first is a multi-class image classifier trained on a custom dataset of 10 object categories. Three architectures were explored: a custom CNN, a minimal Vision Transformer, and a pretrained MobileNetV2 fine-tuned with transfer learning. The second component is the AR rendering pipeline, which uses a pinhole camera model and OpenCV's `projectPoints` function to map 3D box corners into pixel coordinates on each frame.

The main contributions of this work are: (1) a fully functional real-time markerless AR system that runs at 20 to 30 FPS on consumer hardware; (2) a comparative evaluation of three architectures on a small custom dataset; and (3) an analysis of where recognition errors occur and how they affect the AR experience.

II. Related Work

A. Efficient Convolutional Architectures

Sandler et al. introduced MobileNetV2 [1], which uses inverted residual blocks with linear bottlenecks to build an efficient network suitable for mobile and embedded applications. MobileNetV2 is used in this project as the transfer learning backbone because its ImageNet-pretrained weights provide a strong visual feature extractor that generalizes well even when fine-tuned on a small custom dataset.

B. Vision Transformers

Dosovitskiy et al. demonstrated that a pure transformer architecture applied directly to sequences of image patches can achieve strong classification accuracy when trained on large datasets [2]. Their Vision Transformer (ViT) splits an input image into fixed-size patches, linearly embeds each patch, prepends a learnable class token, and passes the sequence through standard transformer encoder layers. The final class token is used for classification. A minimal version of this architecture was implemented in this project to compare against the CNN baseline.

C. Transfer Learning for Image Classification

He et al. introduced deep residual networks (ResNets) [3], showing that very deep networks can be trained effectively using skip connections that allow gradients to bypass layers. ResNets established transfer learning as a standard practice: a network pretrained on ImageNet can be fine-tuned on a new task with far fewer labeled examples than training from scratch would require. This principle directly motivates the use of MobileNetV2 in this project, where the ImageNet-pretrained backbone provides visual features that transfer well to recognizing everyday objects.

D. Pose Estimation for Augmented Reality

Marchand et al. provide a survey of pose estimation methods used in AR systems [4]. They cover feature-based tracking, model-based approaches, and hybrid methods, and discuss the trade-offs between accuracy and computational cost for real-time applications. This work is relevant here because it contextualizes the simplified fixed-depth projection approach used in this system relative to the broader landscape of markerless AR techniques.

III. Methods

A. Dataset

A custom dataset of 10 everyday object classes was assembled. Five classes (book, cup, keyboard, pen, phone) were captured using a custom webcam data collection tool that presents a live center ROI and lets the user capture images by pressing keys. Images were captured with the object at varying distances, angles, and lighting conditions. The remaining five classes (glasses, headphones, laptop, ps5_controller, tablet) were downloaded from the web using the `icrawler` library to search Bing for each class name.

Table I shows the final image counts per class. In total the dataset contains 1419 images. All images were split into 70% training, 15% validation, and 15% test using a fixed random seed of 42 to ensure reproducibility.

TABLE I
DATASET IMAGE COUNTS PER CLASS

Class	Images
book	161
cup	173
glasses	130
headphones	110
keyboard	152
laptop	123
pen	171
phone	159
ps5_controller	102
tablet	138
Total	1419

Training images were augmented with random horizontal flips, random vertical flips ($p = 0.1$), color jitter (brightness ± 0.4, contrast ± 0.4, saturation ± 0.3, hue ± 0.1), random rotations up to ± 25 degrees, random perspective distortion (scale 0.4, $p = 0.5$), random grayscale ($p = 0.1$), and random erasing ($p = 0.3$, area 2–20%). Validation and test images were not augmented.

B. Model Architectures

1) ObjectCNN: ObjectCNN is a three-layer convolutional neural network trained from scratch. It accepts $3 \times 64 \times 64$ RGB images. Each convolutional layer uses a 3×3 kernel with padding 1, followed by batch normalization, ReLU activation, and 2×2 max pooling. The channel depth grows from 32 to 64 to 128 across the three layers. The spatial dimensions shrink from 64 to 32 to 16 to 8. The feature maps are then flattened into a 8192-dimensional vector, passed through a fully-connected layer with 256 neurons and 40% dropout, and finally through an output layer with log-softmax activation. Batch normalization was added to stabilize training on the small dataset.

2) ObjectViT: ObjectViT is a minimal Vision Transformer trained from scratch. The input image is split into 8×8 non-overlapping patches using a strided convolution, yielding 64 patches per image. Each patch is projected into a 128-dimensional embedding space. A learnable class token is prepended to the sequence and learned positional embeddings are added before the sequence enters a stack of four transformer encoder layers, each with 4 attention heads and a feed-forward dimension of 512. The class token output after the final layer norm is passed through a linear classification head.

3) MobileNetV2: A MobileNetV2 backbone pretrained on ImageNet was loaded and its final classification head was replaced with a new linear layer sized for 10 classes, preceded by 30% dropout. The backbone was not frozen: all weights were updated during fine-tuning with a low learning rate. Input images were resized to 224×224 as required by the MobileNetV2 architecture. This model was selected as the final deployed backbone because it consistently outperformed the from-scratch models on the 10-class dataset.

C. Training

All models were trained for 20 epochs using the Adam optimizer with an initial learning rate of 10^{-3} and a cosine annealing schedule that decays the rate to near zero by the final epoch. The loss function was negative log likelihood applied to the log-softmax outputs. The checkpoint with the highest validation accuracy was saved and reloaded for final evaluation.

D. AR Rendering Pipeline

The live AR system processes one webcam frame at a time. A square region of interest (ROI) centered in the frame is cropped and resized to the input dimensions required by the deployed model. Before drawing any overlay, two checks are applied. First, a temporal smoothing buffer retains the last five predicted class indices and takes the majority vote, which prevents the label from flickering between frames when the classifier is close to a decision boundary. Second, an entropy-based rejection step computes the normalised Shannon entropy of the softmax probability distribution:

$$H_{\text{norm}} = \frac{-\sum_i p_i \log p_i}{\log N} \tag{1}$$

where N is the number of classes. When H_{norm} exceeds 0.85 the frame is labelled "Unknown," indicating that no object the model has seen before is present. This removes the need for a dedicated background class while still allowing the system to abstain on unfamiliar scenes.

If the top class passes both checks and exceeds the confidence threshold, the 3D overlay is drawn. The overlay uses a

fixed-depth pinhole camera model. The camera intrinsic matrix is estimated from the frame dimensions:

$$K = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad f = 0.85 \cdot \max(w, h) \tag{2}$$

where w and h are the frame width and height, and c_x, c_y are the principal point at the image center.

The object is assumed to sit at a fixed depth $Z = 0.5\,\mathrm{m}$. This is a deliberate simplification: the original proposal explored monocular depth cues, but a fixed depth was adopted to keep the pipeline real-time without adding a second inference model. From the pixel dimensions of the ROI and the pinhole equation, the physical width W and height H of the bounding box are computed as:

$$W = \frac{w_{\mathrm{ROI}} \cdot Z}{f}, \quad H = \frac{h_{\mathrm{ROI}} \cdot Z}{f} \tag{3}$$

Box depth is set to $D = 0.45 \cdot \min(W, H)$. The 8 corners of this 3D box are projected to pixel coordinates using `cv2.projectPoints` with identity rotation and translation. Because the object is assumed to sit on the optical axis with zero rotation, the back face projects almost concentrically with the front face at this depth; only a slight perspective scaling differentiates them. The 12 edges of the box are drawn as colored lines with each class assigned a unique color, and a semi-transparent label tag showing the class name and confidence score is placed above the front face using alpha blending.

IV. Experiments and Results

A. Evaluation Setup

Each model was evaluated on the held-out test split (15% of all images, fixed seed 42). Overall accuracy and per-class precision and recall were computed. The CNN was trained and evaluated on the original 5-class dataset. The ViT and MobileNetV2 were trained on the full 10-class dataset. Live inference speed was measured as an exponential moving average of frame processing time during the live demo.

Table II summarizes the three architectures.

TABLE II
Architecture Comparison

Model	Classes	Parameters	Test Accuracy
ObjectCNN	5	2.19M	79.7%
ObjectViT	10	0.83M	43.2%
MobileNetV2	10	2.24M	**88.8%**

B. CNN Results (5 Classes)

The CNN trained on the 5-class dataset reached a test accuracy of 79.7% (Fig. 2, Fig. 3). The best checkpoint was saved at epoch 14. The gap between validation and test accuracy is primarily a product of small split sizes. With roughly 120 samples in each held-out partition, a single random seed can produce a test split that is slightly easier or harder than the validation split, shifting results by a few points.

C. ViT Results (10 Classes)

The Vision Transformer trained from scratch on the 10-class dataset reached 43.2% test accuracy (Fig. 1). Three classes, glasses, headphones, and ps5_controller, received near-zero precision, indicating the model did not learn discriminative features for those categories. The mean epoch time was 14.2 seconds on Apple MPS. The result confirms the well-known finding that ViTs trained from scratch on small datasets struggle because transformers require substantial data to learn useful attention patterns.

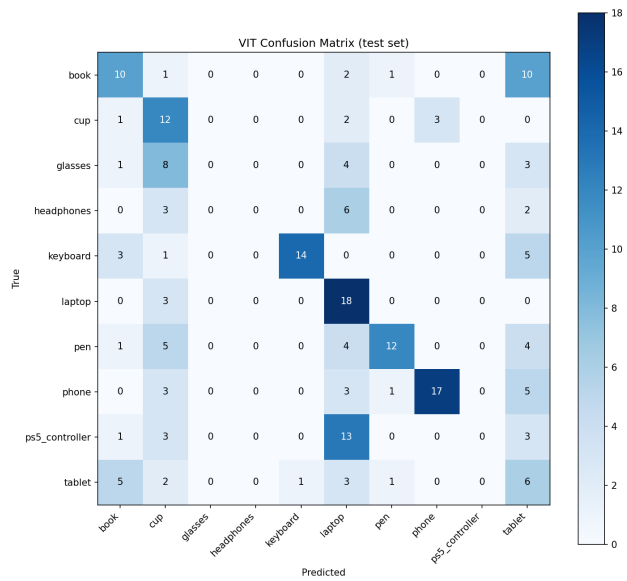

Fig. 1. ViT confusion matrix on the held-out test set (10 classes). Many classes cluster in a few bins, reflecting the model's difficulty generalising from scratch on a small dataset.

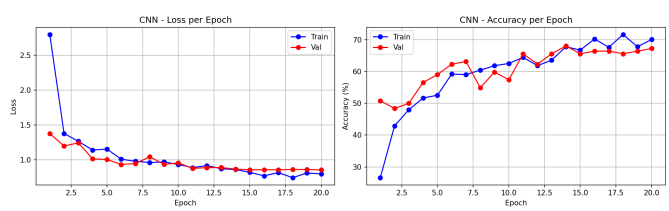

Fig. 2. CNN training and validation loss and accuracy curves (5 classes).

D. MobileNetV2 Results (10 Classes)

The MobileNetV2 model trained on the full 10-class dataset achieved a test accuracy of 88.8%. Training and validation accuracy improved steadily through the first 7 epochs and then leveled off (Fig. 4). Mild overfitting is visible in the later epochs where training accuracy continues to climb past 97% while validation accuracy plateaus near 88%.

The confusion matrix (Fig. 5) reveals which classes the model handles well and which it struggles with. Table III summarizes per-class performance.

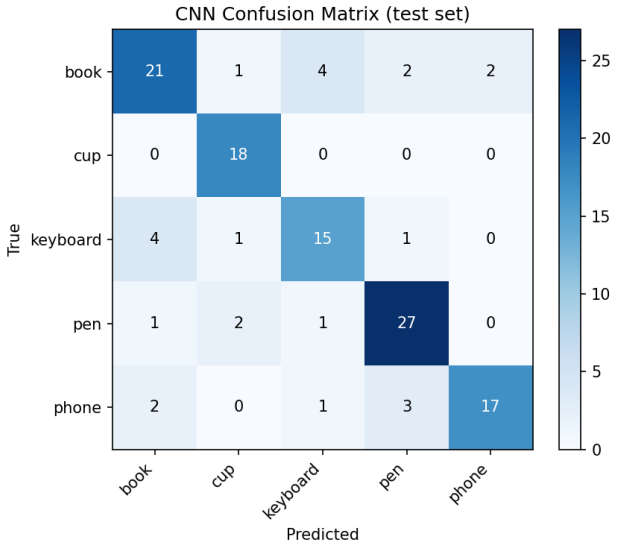

Fig. 3. CNN confusion matrix on the held-out test set (5 classes).

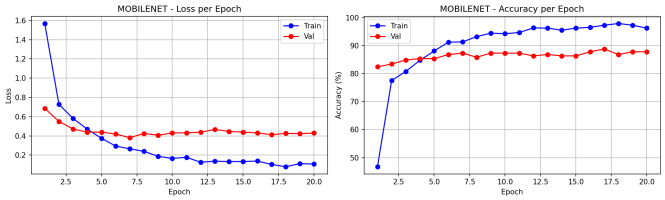

Fig. 4. MobileNetV2 training and validation curves (10 classes, 20 epochs).

The results show strong performance across most classes. Cup and keyboard both achieved 100% recall, and headphones reached 100% precision. Glasses improved dramatically to 87.5% precision and recall under MobileNetV2, compared to near-zero under the from-scratch ViT, confirming that the pretrained backbone learned to distinguish eyeglasses despite the noisy web crawl data. Tablet remained the weakest class with 70% precision and 77.8% recall, as flat rectangular devices are inherently difficult to separate at certain angles.

E. Live AR Performance

The complete inference and rendering pipeline ran at approximately 20–30 FPS during the live demo on an Apple M-series chip with MPS acceleration enabled, varying with scene complexity. This is well above the 15 FPS minimum required for a smooth AR experience. Fig. 6 shows a screenshot captured during the live demo.

V. Discussion and Summary

Transfer learning with MobileNetV2 proved to be the most practical approach for building a real-time object classifier on a small custom dataset. The pretrained ImageNet backbone provides general visual features that carry over well to everyday objects even with only 100 to 170 training images per class, reaching 88.8% test accuracy across 10 classes. Training from scratch with the CNN achieved 79.7% on

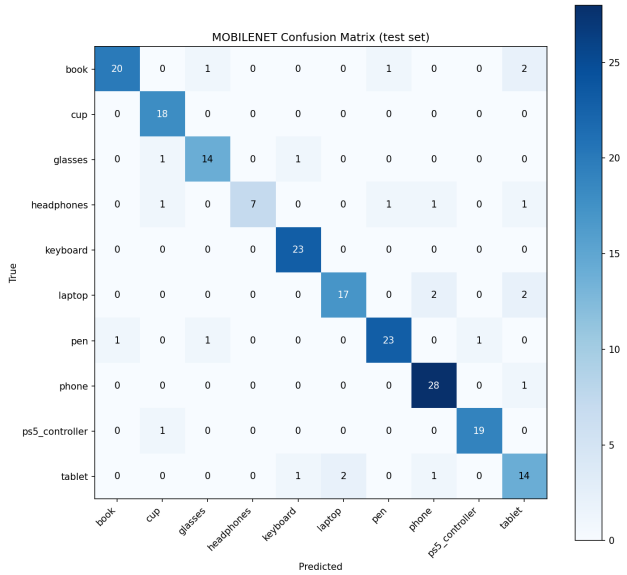

Fig. 5. MobileNetV2 confusion matrix on the held-out test set (10 classes).

TABLE III
MobileNetV2 per-class test results (10 classes)

Class	Precision	Recall
book	95.2%	83.3%
cup	85.7%	100.0%
glasses	87.5%	87.5%
headphones	100.0%	63.6%
keyboard	92.0%	100.0%
laptop	89.5%	81.0%
pen	92.0%	88.5%
phone	87.5%	96.6%
ps5_controller	95.0%	95.0%
tablet	70.0%	77.8%
Mean	89.4%	87.3%
Overall Accuracy	88.8%	

the 5-class dataset, which is reasonable given the aggressive augmentation pipeline but is otherwise limited by the small number of classes. The ViT reached only 43.2% on the 10-class dataset when trained from scratch, consistent with the literature finding that transformers require significantly more data than CNNs to learn useful representations. The parameter count (0.83M for ViT versus 2.19M for the CNN) was not the limiting factor; the lack of pretrained features was.

The biggest remaining source of error is the tablet class at 70% precision, as flat rectangular devices are inherently ambiguous at certain angles and distances. Most other classes performed well, with glasses recovering strongly to 87.5% despite the noisy web crawl data that initially caused confusion between eyeglasses and drinking glasses.

The AR system behaved well within its design constraints. The fixed-depth assumption can cause the 3D box to appear slightly too large or small at unusual distances, but the overlay looks convincing in a normal desktop scenario. Fig. 6 shows a

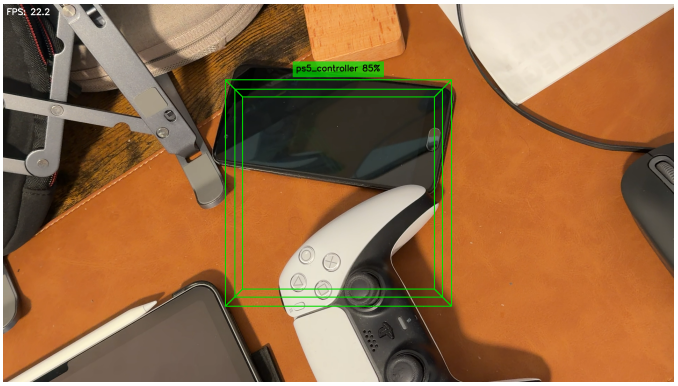

Fig. 6. AR overlay during a live demo session. A PS5 DualSense controller is in frame and the model predicts "ps5_controller" at 85% confidence. The green wireframe box is correctly anchored to the object with the label tag visible above the front face.

representative live frame where the PS5 controller is correctly identified and the wireframe box is cleanly anchored to the object. Even in cases where the classification is wrong, the rendering pipeline remains robust: the box and label are always drawn at the correct position and scale relative to the center ROI.

Two refinements were implemented to improve the live experience beyond the basic pipeline. A five-frame majority-vote buffer removes the label flicker that occurs when the model is near a decision boundary. An entropy-based rejection step flags frames with a highly uncertain probability distribution as "Unknown," giving the system a way to abstain when no familiar object is in view without requiring a dedicated background class.

The remaining limitation worth addressing is the fixed center ROI. Replacing it with a sliding-window or learned detector would allow the system to handle objects anywhere in the frame and support multiple objects at once.

Overall this project produced a working real-time markerless AR system that met and extended the original goals. The pipeline recognizes 10 everyday objects at 20 to 30 FPS and renders a 3D wireframe overlay with no printed calibration target required. The work builds on the AR pipeline from Project 4 and the deep learning experiments from Project 5, connecting them in a way that makes each more useful than it was on its own.

References

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] E. Marchand, H. Uchiyama, and F. Spindler, "Pose estimation for augmented reality: A hands-on survey," *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 12, pp. 2633–2651, 2016.